README | Seguimiento de Agentes

Control de robots físicos Pololu 3Pi con poses de Robotat por MQTT, comandos de ruedas por TCP y visualización en vivo. Un tracker compartido recibe las poses; cada agente ejecuta su control en un hilo.

Archivos

Módulo	Función
main.py	Configura ROBOTS, tracker, hilos y gráfica. Punto de entrada.
leader_controller.py	Modos point y circle; publica objetivos y trayectoria.
follower_controller.py	Persigue la posición actual de follow_agent_id.
robotat_tracker.py	Recibe MQTT y devuelve pose (x_mm, y_mm, theta_rad).
mqtt_robotat.py	Guarda el último paquete y un historial circular por agente.
mqtt_cb.py	Callback auxiliar para un solo pid; no lo usa el tracker actual.
Pololu3Pi.py	Conexión TCP, comandos CBOR y parada de ruedas.
unicycle_utils.py	Cinemática, saturación, ángulos y calibración de ruedas.
visualize_trajectory.py	Gráfica, sliders, pausa y capturas SVG.

Preparación y ejecución

Guardar los nueve archivos en la misma carpeta y quitar los sufijos de descarga (1) y (2), conservando los nombres de la tabla y las mayúsculas de Pololu3Pi.py. Los imports esperan esos nombres.

Instalar las dependencias en el entorno Python que se utilizará:

```
python -m pip install "paho-mqtt>=2" numpy scipy matplotlib
```

La API MQTT usa CallbackAPIVersion.VERSION2. Configurar el acceso a la red Robotat, comprobar los marcadores y revisar IP e identificadores en ROBOTS. Iniciar desde esa carpeta:

```
python main.py
```

Configuración activa de main.py

ID / rol	IP	Sigue a
12 / leader, modo point	192.168.50.112	2
8 / follower	192.168.50.108	12
2 / follower	192.168.50.102	8

La configuración forma el ciclo 12 -> 2 -> 8 -> 12. El rol leader habilita opciones de control, pero en este caso persigue a otro agente; no genera una trayectoria circular. No hay dos entradas activas con el mismo agent_id.

Para ampliar el sistema, añadir una entrada a ROBOTS con role, agent_id e ip. Los followers requieren follow_agent_id. main.py solicita al tracker todos los IDs propios y seguidos; el ID debe corresponder al marcador rastreado.

Modos, parámetros y control

Valores efectivos al ejecutar main.py

Parámetro	Leader: ID 12	Followers: IDs 8 y 2
v0	250 mm/s	250 mm/s
alpha	0.0007	0.0001
kpO / kiO / kdO	2.7 / 0 / 0.55	2.7 / 0 / 0.55
Umbral de parada	eP_stop = 100 mm	eP_stop_fw = 100 mm
Límite de rueda	300 rpm	300 rpm
control_dt	0.1 s	0.1 s
Offset inicial de orientación	100 grados	100 grados

RUN_TIME_S = None permite ejecución indefinida y SHOW_LIVE_PLOT = True abre la gráfica. ROBOTS sobrescribe los valores por defecto; los parámetros propios de followers llevan _fw.

Modos del líder

point: persigue target_point o la pose de follow_agent_id. Frena dentro de eP_stop, reinicia el control angular y retoma si el objetivo se aleja. Los followers usan este seguimiento de posición, sin desplazamiento d_sep.

circle: genera el círculo a la izquierda del heading inicial calibrado. Valores: Ra = 450 mm, NV = 2, tf = 30 s, lookahead_samples = 15 y ePtol = 40 mm. Avanza por tolerancia; tf no detiene la prueba. eP_stop no se aplica en este modo.

Ley de control

La orientación se calibra con theta_offset_deg en grados; theta y eO están en radianes. Para followers se usan los parámetros equivalentes _fw:

```
theta = theta_sign*theta_raw + math.radians(theta_offset_deg)
eP = math.hypot(xg - posx, yg - posy)
eO = wrap_to_pi(math.atan2(yg - posy, xg - posx) - theta)
v = v0*(1 - math.exp(-alpha*eP**2))
```

En point y en los followers se aplica además la reducción por orientación; en circle no se aplica:

```
v *= max(0.0, math.cos(eO))
```

La velocidad angular utiliza suma y diferencia de errores por muestra, sin factores dt ni 1/dt. Con kiO = 0, la acción configurada es PD:

```
eO_D = eO - eO_prev
EO += eO
w = kpO*eO + kiO*EO + kdO*eO_D
```

Ruedas y calibración

inverse_kinematics_rpm usa r = 32 mm y L = 82.4 mm, convierte a rpm y satura cada rueda:

```
q_left  = (2*v - w*L)/(2*r)
q_right = (2*v + w*L)/(2*r)
rpm = q*(60/(2*math.pi))
```

send_wheel_cmd aplica left_sign, right_sign y swap_wheels por robot; no invierte ambos signos de forma fija como en Webots. Pololu3Pi impone un segundo límite de +/-400 rpm, aunque se configure un límite mayor en el controlador.

Comunicación y operación

Robotat: recepción MQTT

Configuración en robotat_tracker.py	Valor
ROBOTAT_HOST / ROBOTAT_PORT	192.168.50.200 / 1880
ROBOTAT_TOPIC	mocap/all
Formato principal	identifier y payload.pose
Formato alternativo	pid y pld.pose
Pose publicada al control	x e y en mm; yaw en radianes.
Historial activo	100 paquetes por agente.

El tracker distribuye cada mensaje a su MqttRobotat según el ID. Obtiene posición y cuaternión [qx,qy,qz,qw], convierte metros a milímetros y extrae yaw con la secuencia ZYX de SciPy. El historial conserva [x,y,z,qx,qy,qz,qw] en las unidades del paquete original; get_history_matrix() devuelve una matriz n x 7.

mqtt_cb.py es un auxiliar de filtrado por pid para un solo marcador. RobotatTracker utiliza su propio callback para admitir varios IDs y el campo identifier.

Pololu 3Pi: envío TCP

Cada hilo abre una conexión a la IP de su robot, puerto 9090. Pololu3Pi envía un arreglo CBOR [rueda_izquierda, rueda_derecha] con dos float32, en rpm. disconnect() intenta mandar cero a ambas ruedas antes de cerrar el socket. La conexión MQTT solo obtiene poses; los comandos de motor viajan por TCP.

Gráfica en vivo

La ventana muestra posiciones, estelas de hasta 300 puntos, objetivos con estrella y puntos de referencia. Se actualiza cada 50 ms y usa ambos ejes invertidos, de 2200 a -2200 mm.

Control	Acción
Slider theta_offset	Ajuste global de -180 a 180 grados; afecta a todos los agentes.
Slider Ra	Solo aparece para líderes en circle: 100 a 1500 mm.
Espacio	Pausa o reanuda los robots; reinicia sus estados angulares al pausar.
S	Guarda una captura vectorial SVG en capturas/.
Cerrar ventana / Ctrl+C	Solicita detener los hilos y desconectar el tracker.

El radio se recalcula manteniendo el centro inicial del círculo. La pausa no cuenta para run_time_s. En modo point, la estrella sigue el objetivo actualizado; la nube de puntos puede conservar una posición anterior si no cambia traj_version.

Detalles a tener en cuenta

Se esperan poses iniciales hasta 15 s. Si el tracker devuelve None durante el control, el robot recibe force_stop(). El tracker conserva el último paquete sin comprobar su antigüedad: perder mensajes no garantiza que la pose pase a None ni que el robot se detenga.

No se implementan prevención de colisiones ni anti-windup general. La captura SVG no sustituye un registro completo: los historiales MQTT están en memoria y no se exportan automáticamente. La opción python visualize_trajectory.py solo visualiza; usa una configuración independiente con ID 5 y no inicia los controladores de main.py.